

DOM-Based Participant Attendance Tracking for Google Meet Breakout Rooms: A Comparison with OCR-Based Screen Reading

Harsha Sunkavalli

Department of Computer Science

George Mason University

smasriharsha@gmail.com

Abstract—Taking attendance in video meetings that use breakout rooms is tedious to do by hand, and the official reporting available to a host does not always fit the need. An earlier tool addressed this by taking screenshots of the Google Meet participant panel and recovering names with optical character recognition (OCR). That design is exposed to every property of the rendered pixels: fonts, scripts, zoom, layout and overlays. This paper presents a replacement that reads participant names and breakout-room membership directly from the page’s Document Object Model (DOM) through browser automation, and feeds them to a session engine that tolerates transient read errors with a grace period and back-dated leave times. We compare the two methods by running the *real* implementation of each on identical mock Meet-style panels in headless Chromium, over 19 test cases (12 standard, 7 adversarial) containing 311 participant slots. The DOM tracker recovered 311 of 311 names exactly (100.0%), produced 0 spurious names, and placed 100.0% of names in the correct room. The OCR tracker recovered 165 (53.1%), produced 124 spurious names, and placed 39.5% correctly. On a 60-person list the DOM scan was $3.4\times$ faster. A seeded simulation shows that a two-scan grace period reduces split attendance records from 45.8% to 3.0% at a 5% per-scan read-miss rate. The evaluation uses mock pages, not a live Meet call, and the OCR implementation was tuned on real screens, so its scores here likely understate its field accuracy; we discuss these limits and give a first-run procedure for validating the DOM selectors on live meetings.

Index Terms—attendance tracking, Google Meet, breakout rooms, DOM extraction, optical character recognition, browser automation, web data extraction

I. INTRODUCTION

Online meetings with breakout rooms are common in teaching, training and community organizations, and organizers often need to know who attended, in which room, and for how long. Doing this by hand during a session is slow and error-prone, particularly when a participant list has dozens of entries and people move between the main call and several rooms. The original motivation for this work was a volunteer organization that held weekly Google Meet sessions with breakout rooms; its tool, written by the author, replaced manual head counts with timestamped CSV reports and, as reported by the author, saved several hours of administrative work per session. That figure is anecdotal and is not measured in this paper.

The original tool is *vision based*. It opens the breakout-rooms panel under browser automation, takes screenshots

while scrolling, finds the panel’s left edge by pixel colour analysis, runs Tesseract OCR [1] over three image variants, groups recognized words into lines, infers room headers from text patterns, splits side-by-side name chips by horizontal gaps, filters recognition noise with regular expressions, and detects presence dots with colour masks in OpenCV [2]. This works, but each stage encodes assumptions about appearance. Names that contain accents, non-Latin scripts or emoji are hard for the recognizer; a different zoom, theme or layout can move the panel edge; and a mid-scan layout change must be detected and discarded. Errors are *silent*: the output is a plausible but wrong name.

The page that is being photographed already contains the answer in structured form. The participant list is part of the DOM [3], and Meet, like other web applications, exposes much of its UI through accessibility attributes defined by WAI-ARIA [4]. This paper asks whether reading that structure instead of the pixels yields a more accurate, faster and more maintainable attendance tracker, and what new risks it introduces.

Contributions.

- 1) A DOM-based tracker design for Meet attendance, including breakout-room assignment from document order, scroll-and-merge for virtualized lists, and a layered architecture in which only one component touches the browser.
- 2) A session engine with a grace period and back-dated leave times, and a fail-safe rule that an empty read is never treated as “everyone left”.
- 3) A head-to-head evaluation that runs the real implementations of both methods on identical pages: 19 cases, 311 participant slots, reported with precision, recall, F1, room accuracy and scan time, plus a seeded simulation of the grace-period trade-off.
- 4) An account of defects found by the verification process itself, and an explicit statement of what the evaluation does *not* establish.

The source code, benchmark harness, raw results and this paper are in the project repository [5].

The remainder of the paper is organized as follows. Section II reviews background. Section III formalizes the task. Sections IV and V describe the baseline and the proposed system. Section VI gives the experimental method, Section VII

the results, and Section VIII discusses limits and ethics. Section IX concludes.

II. BACKGROUND AND RELATED WORK

A. Attendance in Google Meet

Meet’s participant panel lists who is in the call and, for a host using breakout rooms, which room each person is in [6]. Official programmatic interfaces and administrator reports exist for some account types; their availability and the data they expose depend on the account and edition. This work follows the original tool’s constraint of needing no API access, using only the signed-in host’s browser, and we did not evaluate official interfaces.

B. OCR and screen reading

OCR converts images of text into character sequences and has a long history [7]. Tesseract [1] is a widely used open-source engine. Accuracy depends on resolution, contrast, font and script, and the output is a best guess with a confidence score rather than a ground-truth string. Using OCR on a rendered user interface inherits these limits and adds layout analysis: deciding which recognized words form one name, which are decoration, and which belong to which section.

C. Web data extraction and the DOM

Web data extraction is the problem of recovering structured data from web pages; surveys distinguish approaches that rely on page structure from those that rely on rendered appearance [8]. Structure-based extraction is attractive when markup is stable, but brittle when it is not. Browser automation interfaces such as WebDriver [9] (implemented by Selenium [10]) and Playwright [11] let a program evaluate JavaScript inside the page and return values. WAI-ARIA [4] defines roles (`list`, `listitem`, `heading`) and properties (`aria-label`, `aria-expanded`) that exist for assistive technology and are therefore more stable than the generated CSS class names typical of large web applications. Our selector strategy is built on that observation.

D. Evaluation measures

We use precision, recall and F1 [12], [13], defined in Section VI. Fuzzy string similarity, used only to count near-miss recognitions, is based on the Ratcliff–Obershelp gestalt matching algorithm [14].

III. PROBLEM FORMULATION

Let a meeting be observed at scan times $t_1 < \dots < t_K$ separated by an interval Δ (300s in real use). At scan k the visible panel yields a snapshot

$$S_k = \{(r, n)\}, \quad r \in \mathcal{R}, \quad n \in \mathcal{N}, \quad (1)$$

a set of (room, display-name) pairs, where the main call is a room. A tracker is a pair $(\mathcal{E}, \mathcal{A})$: an *extractor* $\mathcal{E}$ that maps the page to a snapshot $\hat{S}_k$, and an *attendance engine* $\mathcal{A}$ that maps the snapshot sequence to sessions $(r, n, t_{\text{join}}, t_{\text{leave}})$. The report is the set of sessions.

We want an extractor that (i) recovers names exactly, so that reports can be matched to rosters; (ii) assigns each name to the right room; (iii) emits few or no spurious names; (iv) fails visibly; and (v) is cheap to run and to maintain. We want an engine whose sessions are not fragmented by isolated read errors and whose durations are accurate to within roughly one scan interval. The tracker’s own account is excluded from the report.

IV. BASELINE: OCR PIPELINE

The baseline extractor $\mathcal{E}_{\text{OCR}}$ proceeds as follows for each scroll position of the panel:

- 1) Capture a screenshot and detect the panel’s left edge by scanning columns for the dark panel background, then crop.
- 2) Upscale by two and run Tesseract on three variants (contrast-adjusted, inverted, and CLAHE-enhanced), merging words that fall in the same position bucket.
- 3) Group words into lines by vertical position; derive room headers from text (“Main call”, “Breakout N Join”, lines containing “room”).
- 4) Assign each remaining line to the nearest header above it; split lines into names where horizontal gaps are large; remove strings that match noise phrases, fragments, or lowercase-initial text.
- 5) For each name, crop a region of interest and test for a green or yellow circular blob in HSV space as a presence cue.
- 6) Merge scroll positions, treating the first screenshot’s room list as canonical so that rooms first seen after scrolling are ignored, and discard the whole scan if screenshot widths differ by more than 100px (layout change).

The method embodies many tuned constants and text patterns. Its strengths are that it needs no knowledge of page internals and that it observes what a human would see.

V. PROPOSED SYSTEM: DOM-BASED TRACKER

A. Architecture

Fig. 1 shows the layers. Only the *Panel reader* interacts with the browser. Everything after it is plain Python, which makes the engine, normalization and report writer testable without a browser. A scan loop runs in its own thread and a stop event ends it and flushes reports.

B. Locating and scrolling the panel

The reader finds visible `role="list"` elements that contain list items, walks up to the nearest scrollable ancestor, and scrolls it from top to bottom in steps of 80% of its height, extracting at each step and de-duplicating by (room, raw name). This handles *virtualized* lists, which keep only visible rows in the DOM. A bounded number of steps guards against runaway loops. If the panel is closed the reader tries to open it from the toolbar by accessible label.

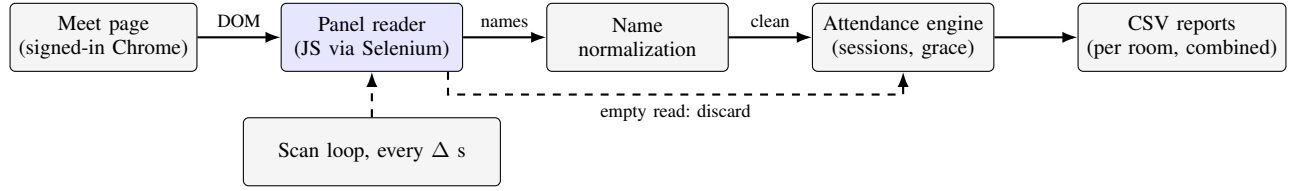


Fig. 1. DOM tracker architecture. The shaded component is the only one that touches the browser; the engine discards empty reads so a failed scan never ends sessions.

Algorithm 1 Room-aware extraction (runs in the page)

```

1:  $room \leftarrow \text{MAINCALL}$ ;  $out \leftarrow []$ 
2: for each element  $e$  under the panel, in document order do
3:   if  $e$  has role listitem then
4:      $name \leftarrow$  first non-empty of: data attribute,
       aria-label, nested aria-label, first text line
5:     append  $(room, name)$  to  $out$ 
6:   else if  $e$  is not inside a listitem and is a heading,
       h2/h3, or has aria-expanded then
7:      $txt \leftarrow$  first line of  $e$  without trailing “(N)” and
       “Join”
8:     if  $txt \neq \emptyset$ ,  $|txt| < 60$ , and  $txt \notin \text{NOTROOM}$  then
9:        $room \leftarrow \text{MAINCALL}$  if  $txt$  matches “main call”;
       else  $txt$ 
10:    end if
11:  end if
12: end for
13: return  $out$ 

```

C. Structural extraction

Algorithm 1 gives the extraction walk, executed in the page. It relies on document order: a heading-like element starts a new room section, and every list item belongs to the current section.

Two design choices matter. First, room names are not matched against a naming pattern: any heading outside a participant row that is not a known section title (“People”, “Contributors”, “Breakout rooms”, and similar) is a room, so custom names work. Second, headings inside a participant row are ignored, because rows commonly contain buttons with `aria-expanded` (for example a “more actions” menu) that would otherwise be misread as rooms. The first rule replaced an earlier pattern-based version that failed on headings such as “Breakout 1” (Section VII-E).

D. Name normalization

Raw strings from the DOM are normalized before use: zero-width characters are removed, whitespace is collapsed, trailing decorations such as “(You)”, “(Host)” and “(Presentation)” are stripped (repeatedly), and full Meet phrases such as “Pin X to your main screen” and “More actions for X ” are unwrapped. Strings that are UI words (“Mute”, “Host”, “Contributors”) or contain no letters are rejected. Names are keyed case- and whitespace-insensitively. Only *full phrases* are unwrapped, not bare verbs, so a participant named “Pin Sharma” is preserved.

Algorithm 2 Engine update for one room and scan at time t

```

1:  $V \leftarrow \{\text{KEY}(n) : n \in \text{normalized names}, n \neq \text{self}\}$ 
2: for  $v \in V$  do
3:    $miss[v] \leftarrow 0$ ;  $last[v] \leftarrow t$ 
4:   if  $v \notin present$  then
5:     open session  $(v, t_{\text{join}}=t)$ ; add  $v$  to  $present$ 
6:   end if
7: end for
8: for  $v \in present \setminus V$  do
9:    $miss[v] \leftarrow miss[v] + 1$ 
10:  if  $miss[v] \geq g$  then
11:    close session with  $t_{\text{leave}} \leftarrow last[v]$ ; remove  $v$ 
12:  end if
13: end for

```

E. Attendance engine

Algorithm 2 summarizes the update rule. A participant is considered to have left only after g consecutive scans without them (default $g = 2$), and the recorded leave time is back-dated to the last scan at which they were seen. At shutdown, anyone who was already missing is closed at their last-seen time rather than at the stop time; all others are closed at the stop time.

F. Failure handling and diagnostics

If the panel is closed and cannot be reopened, or the extraction returns nothing (for example because selectors went stale), the scan is skipped and the engine receives no update. This prevents a failure from being recorded as everyone leaving. A diagnostic mode captures the panel’s HTML and the parsed result for a live meeting, so that selectors can be adjusted when Meet’s markup changes. All Meet-specific selectors are isolated in a small set of JavaScript constants.

G. Implementation

The tracker is a single Python file of 484 lines (versus 1,072 for the OCR tool) depending on Selenium,undetected-chromedriver and pandas; it needs no image-processing library and no OCR binary. It writes the same CSV schema as the baseline (room, name, join time, leave time, duration in minutes, date), one combined file and one per room.

VI. EXPERIMENTAL METHOD

A. Mock-panel harness

A live Meet call could not be used for this study, so each test case is a mock panel that imitates Meet’s side panel

(dark theme, rows with avatar and name, section headings) rendered in headless Chromium at 1366×768 . For each case the *same page* is given to both trackers: the DOM tracker’s real `PanelReader` runs its JavaScript in the page through an adapter that mimics the Selenium script-execution interface, and the OCR tracker’s real code (screenshot cropping, panel-edge detection, Tesseract 5.3.4, cleaning and merge) processes screenshots of the same page taken at each scroll position. Both outputs are compared with known ground truth. Nothing in either implementation was replaced by a stub apart from the browser-launch dependency of the OCR module.

B. Test cases

Table II lists the 19 cases. The 12 standard cases vary size (5, 30 and 60 people), the number of rooms, name difficulty (accents, Chinese, Cyrillic, emoji, apostrophes, hyphens, lowercase), the markup style (h2 headings, aria-expanded headings, rows without aria-label, nested aria-label), empty rooms, and a chip layout in the style the OCR code was written for (S11, S12). The 7 adversarial cases target the DOM method specifically: decorated labels with the tracker’s own row (A01), per-row action buttons with aria-expanded (A02), non-room section titles (A03), custom and non-English room names (A04), a virtualized 60-row list (A05), one person listed twice (A06) and a closed panel (A07).

C. Metrics

Names are compared case- and whitespace-insensitively. Let T be the set of true names, G the set output, and $D = T \cap G$. $Recall = |D|/|T|$; $precision = |D|/|G|$; F_1 is their harmonic mean; *room accuracy* is the fraction of T that were reported in the correct room (rooms are compared after canonicalization, so “Main call” equals “Main Call” and “Breakout 1” equals “Breakout Room 1”). *Spurious* names are $G \setminus T$. A case *passes* only if recall, precision and room accuracy are all 100%. Aggregate figures pool all slots; A07 has no participants and contributes only to spurious counts and pass/fail.

D. Grace-period simulation

To study the engine in isolation we simulate 50 participants over a 90-minute meeting with scans every 5 minutes (19 scans). Half stay throughout and half leave permanently at a random scan. At each scan every present participant is independently missed with probability $p \in \{0, 1, 2, 5, 10\}\%$, modelling transient read errors. For $g \in \{1, 2, 3\}$ we run 300 meetings per cell with fixed seeds, using the real engine, and record the fraction of participants whose attendance is split into more than one session and the mean absolute error of total recorded minutes against truth.

E. Unit tests

Twenty-five offline tests cover name normalization (Unicode, emoji, decorations, noise words, wrapped phrases), the engine (grace values, rejoin, room moves, self exclusion, duplicates, shutdown back-dating, 100-person roster, concurrent access), the CSV schema, and the failed-scan safety rule.

TABLE I
AGGREGATE ACCURACY OVER 19 CASES AND 311 PARTICIPANT SLOTS

Metric	DOM	OCR
Cases passed (all names, no extras, right rooms)	19/19	2/19
Recall (%)	100.0	53.1
Precision (%)	100.0	57.1
F1 (pooled)	1.000	0.550
Room accuracy (%)	100.0	39.5
Spurious names output	0	124
Total scan time, all cases (s)	15.4	32.1

VII. RESULTS

A. Aggregate accuracy

Table I gives the pooled results. The DOM tracker passed all 19 cases; the OCR tracker passed 2. DOM recovered every name exactly and placed every name in the correct room with no spurious output. OCR recovered 165 of 311 names, output 124 names that were not real, and placed 123 names in the correct room.

B. Per-case results

Fig. 2 and Table II show the cases. OCR was accurate on the smallest baseline (S01) and on the most benign adversarial cases but degraded with list length, name difficulty and markup variation. On the 60-person lists it recovered 21 of 60 (S04) and 29 of 60 (A05) names and output 31 and 30 spurious names. On the hard-name case (S05) it recovered 1 of 10. On the chip layout built to resemble its own conventions (S11, S12) and on the duplicated-row case (A06) it recovered none; on inspection, its S11 output consisted of last-name fragments such as “Brown” and “Wilson”.

C. Scan time

Fig. 3 plots the time for one full scan against list size. The DOM scan is dominated by fixed waits between scroll steps (about 0.6s) and grows slowly with list length; OCR time grows with the number of screenshots and recognition passes. At 60 participants, DOM took 1.9s and OCR 6.4s, a ratio of $3.4\times$. Over all cases the totals were 15.4s and 32.1s. These are single-run timings on one machine and are indicative only.

D. Effect of the grace period

Fig. 4 and Table III show the simulation. With $g = 1$ (leave on the first missed scan) split attendance is already 12.3% at a 1% miss rate and reaches 45.8% at 5%, with a mean duration error of 6.48 minutes. With $g = 2$ the same 5% miss rate gives 3.0% split attendance and 0.70 minutes of error; with $g = 3$ it gives 0.2% and 0.29 minutes. Because leave times are back-dated to the last observation, a larger grace period does not inflate durations; its cost is that a real departure is reported by the live log g scans later, and that tracking of someone who rejoins within the window is continuous rather than split. Without read errors ($p = 0$) all settings are exact.

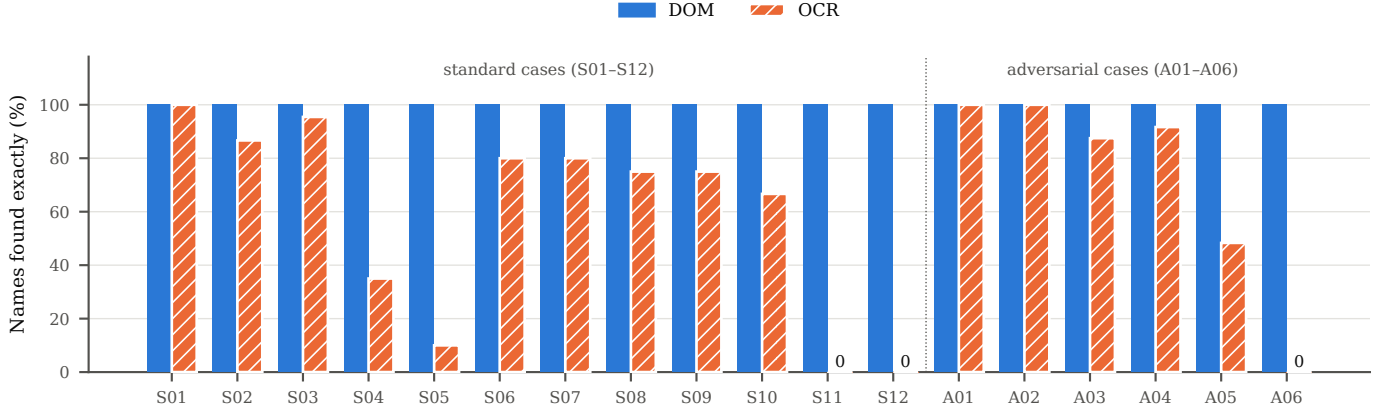


Fig. 2. Names recovered exactly per case (A07 omitted: no participants). A label “0” marks a case where OCR recovered no name exactly.

TABLE II
PER-CASE RESULTS. *Found* = REAL NAMES RECOVERED EXACTLY; *Extra* = OUTPUT NAMES THAT ARE NOT REAL; *Room* = NAMES IN THE CORRECT ROOM. *n* = REAL PARTICIPANTS.

ID	Description	<i>n</i>	DOM		OCR			
			Found	Extra	Found	Extra	Room	Time (s)
S01	Baseline, 5 people, 1 room	5	5	0	5	0	5	0.8
S02	30 people, Main Call	30	30	0	26	11	26	3.6
S03	Main + 3 breakout rooms	22	22	0	21	3	3	2.4
S04	60 people, 3 rooms, scrolled	60	60	0	21	31	15	6.4
S05	Accents, CJK, Cyrillic, emoji	10	10	0	1	5	1	1.0
S06	Headings as <h2>	10	10	0	8	3	4	1.0
S07	Headings via aria-expanded	10	10	0	8	3	4	1.1
S08	Rows without aria-label	8	8	0	6	3	6	0.9
S09	Nested aria-label	8	8	0	6	3	6	0.9
S10	Empty rooms between rooms	6	6	0	4	3	2	0.9
S11	OCR-native chip layout	18	18	0	0	6	0	0.7
S12	OCR-native chips, 30 people	30	30	0	0	13	0	1.4
A01	Decorated labels + own row	6	6	0	6	1	6	0.8
A02	Per-row action buttons	6	6	0	6	0	6	0.7
A03	Non-room section titles	8	8	0	7	3	7	1.0
A04	Custom/non-English room names	12	12	0	11	5	3	1.2
A05	Virtualized list, 60 people	60	60	0	29	30	29	6.3
A06	One person listed twice	2	2	0	0	0	0	0.6
A07	Panel not open	0	0	0	0	1	0	0.6

TABLE III
GRACE-PERIOD SIMULATION (300 MEETINGS PER CELL, 50 PARTICIPANTS, 19 SCANS). CELLS: PARTICIPANTS WITH SPLIT ATTENDANCE (%) / MEAN ABSOLUTE DURATION ERROR (MIN).

Miss prob.	grace=1	grace=2	grace=3
0%	0.0 / 0.00	0.0 / 0.00	0.0 / 0.00
1%	12.3 / 1.37	0.1 / 0.07	0.0 / 0.05
2%	22.8 / 2.70	0.5 / 0.18	0.0 / 0.10
5%	45.8 / 6.48	3.0 / 0.70	0.2 / 0.29
10%	69.7 / 12.81	10.3 / 2.26	1.1 / 0.78

E. Defects found by verification

Running the harness exposed three defects in the DOM tracker, each now fixed and covered by a unit test.

- 1) *Room-heading detection was too narrow.* The first version recognized only headings of the form “Breakout

Room ...” and filed participants under “Breakout 1” or “Group A” in the main call. Detection now accepts any heading-like element outside a participant row that is not a known section title.

- 2) *Name unwrapping stripped bare verbs.* A leading “Pin” or “Mute” was removed from names, turning “Pin Sharma” into “Sharma”. Only full Meet phrases are unwrapped now.
- 3) *Shutdown over-credited departures.* The simulation showed non-zero duration error even without read errors when $g > 1$: at shutdown, a person who had already left but was still inside the grace window was credited up to the stop time. Leave times for such participants are now back-dated to their last observation.

The first two were found by test cases and the third by the simulation, which is an argument for testing the session logic separately from extraction.

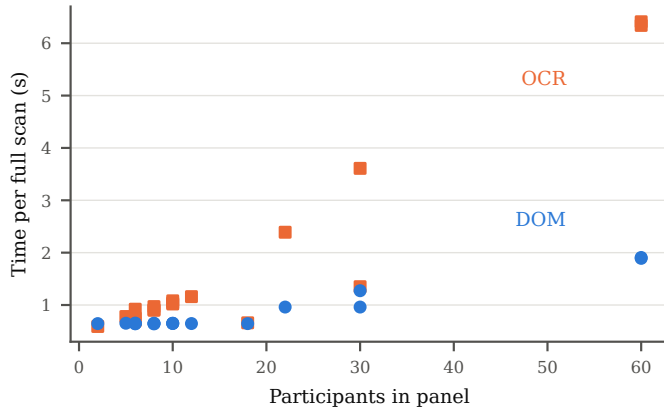


Fig. 3. Time for one full scan versus participants in the panel, one point per case.

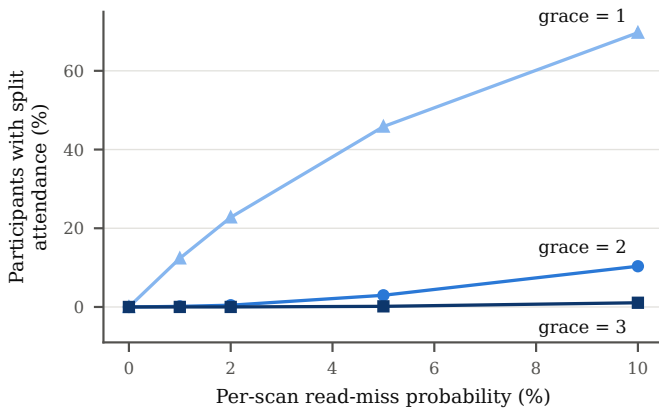


Fig. 4. Participants whose attendance was split into several sessions versus the per-scan read-miss probability, for grace periods of 1, 2 and 3 scans.

VIII. DISCUSSION

A. Why the DOM approach performs better here

Reading text from the DOM removes the recognition step, so names survive exactly, including scripts and symbols that an OCR engine tuned for Latin text handles poorly. It removes layout analysis, so room assignment follows the document structure and custom names work. It removes dependence on pixels, so zoom, theme and window size do not matter. It also changes the failure mode: a stale selector yields an empty read that is detected and skipped, whereas an OCR error yields a wrong name that looks valid.

B. What OCR still offers

OCR needs no knowledge of the page’s internals and therefore survives markup changes that would break DOM selectors. It also works on any application whose content can be screenshotted, where DOM access is impossible. A practical system could use the DOM method as the primary reader and fall back to OCR when selectors return nothing; we did not build or evaluate such a hybrid.

C. Threats to validity

Mock pages. The most important limit is that the pages are mocks. They follow Meet’s accessibility structure as understood by the author, but the real markup may differ, particularly in how breakout-room headings and rows are exposed. The 100% DOM result shows that the extraction logic is robust across many plausible markups; it does not show that the tracker works on Meet today. The provided diagnostic mode and a first-run checklist are intended to close that gap.

OCR is likely understated. The OCR implementation was tuned on real Meet screens, and the mock only approximates real fonts, chips and colours. Its absolute scores here, notably the zero on the chip layout, are therefore best read as sensitivity to layout rather than as production accuracy.

OCR language model. The original tool does not select a Tesseract language, so recognition used the default English model. The very low score on the hard-name case (S05) partly reflects this; installing additional language models might improve non-Latin names, and we did not test that.

Tuning on the benchmark. The DOM extractor was corrected after the first benchmark run exposed a defect, so its final result is partly tuned to these cases. A fresh set of live pages is the real test.

Construct and scope. Exact name matching counts a one-character recognition error as a miss. The simulation assumes independent read errors, whereas real errors are likely correlated in time (for example during a re-render). Only one run per case on one machine was made. Two different participants with an identical display name cannot be told apart by either tracker, and neither case was evaluated.

D. Privacy, consent and platform terms

Attendance data are personal data. Organizers should tell participants that attendance is recorded and handle the CSV files accordingly. The tool automates a signed-in browser session and uses a driver intended to avoid bot detection; users should check that this is consistent with the platform’s terms for their account type and prefer sanctioned interfaces where those meet their needs. The source code stores no credentials; login is manual.

IX. CONCLUSION AND FUTURE WORK

We replaced a vision-based Meet attendance tracker with one that reads participant names and room membership from the DOM and tracks sessions with a grace period. On identical mock panels, the DOM tracker recovered all 311 names with no spurious output and correct rooms in all 19 cases, while the OCR tracker recovered 53.1% of names and 39.5% of room assignments, and the DOM scan was several times faster on long lists. A simulation showed that a two-scan grace period removes most fragmentation caused by transient read errors, and the verification process itself uncovered three defects that were fixed.

The principal next step is validation on live meetings using the provided diagnostic mode. Further work includes

using stable participant identifiers if the page exposes them, so that identical display names can be separated; replacing polling with a `MutationObserver` for event-driven join and leave times; a hybrid DOM-primary, OCR-fallback reader; a comparison with official reporting interfaces where available; and a study over many real sessions with a hand-verified roster.

REPRODUCIBILITY

All code, the harness (`verify_dom_vs_ocr.py`), the simulation (`paper/grace_simulation.py`), the raw results (`benchmark_results.json`, `paper/grace_results.json`) and the scripts that generate this paper’s tables and figures are in the repository [5]. The accuracy, timing and simulation numbers in the text and tables are generated from those result files; other figures (such as source line counts) are stated directly.

REFERENCES

- [1] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.
- [2] G. Bradski, “The OpenCV library,” *Dr. Dobbs Journal of Software Tools*, 2000.
- [3] WHATWG, “DOM standard,” Living Standard, <https://dom.spec.whatwg.org/>.
- [4] W3C, “Accessible rich internet applications (WAI-ARIA) 1.2,” W3C Recommendation, Jun. 2023, <https://www.w3.org/TR/wai-aria-1.2/>.
- [5] H. Sunkavalli, “Google Meet breakout room attendance tracker (source code, benchmark harness, and data),” <https://github.com/futuretide/Google-meet-attendance-tracker>.
- [6] Google, “Google Meet help center,” <https://support.google.com/meet>.
- [7] S. Mori, C. Y. Suen, and K. Yamamoto, “Historical review of OCR research and development,” *Proc. IEEE*, vol. 80, no. 7, pp. 1029–1058, 1992.
- [8] E. Ferrara, P. D. Meo, G. Fiumara, and R. Baumgartner, “Web data extraction, applications and techniques: A survey,” *Knowledge-Based Systems*, vol. 70, pp. 301–323, 2014.
- [9] W3C, “WebDriver,” W3C Recommendation, Jun. 2018, <https://www.w3.org/TR/webdriver1/>.
- [10] Selenium Project, “Selenium WebDriver documentation,” <https://www.selenium.dev/documentation/webdriver/>.
- [11] Microsoft, “Playwright documentation,” <https://playwright.dev/>.
- [12] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [13] D. M. W. Powers, “Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation,” *J. Mach. Learn. Technol.*, vol. 2, no. 1, pp. 37–63, 2011.
- [14] J. W. Ratcliff and D. E. Metzener, “Pattern matching: The gestalt approach,” *Dr. Dobbs Journal*, vol. 13, no. 7, p. 46, Jul. 1988.